Profiling and Architectural Improvement of Contur

Junying Wu

A Dissertation Submitted for the Degree of

Master of Science
of
University College London

Department of Physics and Astronomy
University College London

August 2026

Declaration

I, Junying Wu, confirm that the work presented in this dissertation is my own. Where information has been derived from other sources, I confirm that this has been indicated in the dissertation.

Abstract

Contur is a Python software package for testing predictions from beyond-the-Standard-Model physics against published collider measurements. As its analysis coverage and functionality have grown, both computational cost and package dependencies have become important maintenance concerns. This project investigates these concerns through performance profiling, targeted optimisation and static dependency analysis. Profiling identified unnecessary object construction. Removing it reduced mean execution time by 35.73 % for the recorded workload, while the selected statistical outputs matched exactly across repeated comparisons. The architecture work moved misplaced responsibilities out of low-level packages and produced a provisional automated check that records departures from the intended dependency structure without yet blocking merges. A matched source census recorded 26 conflicting imports after the changes, compared with 14 at the baseline. This increase mainly reflects database dependencies made visible by the new package boundary, which remains unresolved. Together, these results show that targeted profiling and incremental restructuring can improve efficiency and clarify dependencies in mature scientific software.

Contents

Chapter 1

Introduction

Contur is a Python toolkit used by particle physicists to test proposed models of new physics against published collider measurements. For one chosen set of model parameters, Contur processes simulated predictions and calculates how strongly the measurements constrain the model. A parameter scan repeats this analysis for many different sets of values. The official Contur toolkit paper describes these scans as parallel jobs on a compute farm [1]. Any unnecessary operation is therefore repeated many times. This increases the computing time required for a scan and, when computing resources are limited, can reduce the number of parameter values that can be studied.

As Contur has grown, the relationships between its Python packages have also become harder to manage. Some lower-level packages import code from packages that are intended to sit above them, and some groups of imports form loops. These relationships make it harder to decide where a task belongs and to change one part of the program without affecting another.

This dissertation studies these two practical problems. The first part uses profiling to find unnecessary work in the repeated analysis and measures the effect of removing it. The second part examines the imports and responsibilities of selected packages, then moves clearly misplaced code in small changes. It also introduces an automated check that records imports which do not follow the proposed package structure. The dissertation distinguishes changes already merged into the main codebase from work that remains on a separate branch and questions that are still open.

1.1 Problem definition

The performance question is not simply how to make Contur faster. The project must identify a specific operation that is repeated unnecessarily, remove it and check that the relevant results do not change. The timing comparison must also use the same controlled workload before and after the change.

The architecture question is where each responsibility should be placed and which packages should be allowed to import one another. For example, a package intended to provide general utilities should not need to know about higher-level reporting or analysis tasks. Imports in both directions can form loops and make the separation between packages unreliable. This project

does not attempt a complete redesign of Contur. It focuses on a small set of packages involved in configuration, data access, object construction and workflow coordination. The final separation between database access and the objects used by the analysis remains a design question that requires further evidence and agreement from the maintainers.

1.2 Aim and objectives

The project aims to make the studied Contur workflow faster and the codebase easier to maintain. The objectives are:

1. Use a reproducible profiling workflow to identify repeated unnecessary work.
2. Measure the effect of removing that work and compare the selected statistical outputs.
3. Identify unwanted imports and move clearly misplaced code in small, reviewable changes.
4. Use an automated check to record the remaining unwanted imports.
5. Investigate how database queries, analysis objects and their callers should be separated.

1.3 Main results and limitations

The project produced two main results. First, profiling identified an unused object that was being constructed repeatedly. Removing it reduced the mean execution time of three controlled grid runs by 35.73 %, while the selected statistical outputs remained unchanged. Second, the dependency analysis guided several small moves to more appropriate packages, including the merged relocation of webpage reporting out of `util`. A matched source census found that the five original upward imports from `util` had been removed. The total number of imports that conflict with the target structure nevertheless increased from 14 to 26 because the new `database` package made unfinished boundary work visible.

These results have clear limits. The timing result applies only to the controlled workload in chapter 3, and the output comparison covered selected values rather than every possible output. The automated dependency check remains on a separate branch and does not prevent merges. The final separation between database access and analysis objects remains unresolved.

The code, retained experimental evidence and contribution index accompanying this dissertation are available at https://github.com/ucapwun/contur-performance-and-architecture. Production changes are linked there to their authoritative merge requests in the upstream Contur repository.

1.4 Dissertation structure

Chapter 2 explains how Contur works and introduces the relevant Python packages. Chapter 3 presents the profiling and optimisation work. The dependency analysis and implemented

changes are described in chapters 4 and 5. Chapter 6 examines the unresolved separation between database access and analysis objects. Finally, chapters 7 and 8 discuss the evidence, its limitations and the remaining technical work.

Chapter 2

Contur Background and Existing Software Architecture

2.1 Scientific and computational role of Contur

Contur is used in high-energy particle physics, where collider experiments study fundamental particles and their interactions. It tests proposed models of new physics against published collider measurements [1]. Such a model describes possible particles or interactions beyond the Standard Model. Its predictions depend on parameters, which are numerical quantities such as a particle mass or an interaction strength (coupling).

At a high level, the comparison has two inputs. The first is a simulated collider prediction for the proposed model. Its histogram and scatter data are stored using the YODA format provided by the YODA histogramming library [2]. The second is the set of published particle-level measurements preserved in Rivet. Contur reuses these measurements to test whether the model predicts deviations that they disfavour, rather than requiring a dedicated new search for each model [1, 3].

A model-parameter point is one particular choice of values for all the parameters being studied. Contur compares the simulated prediction at that point with the relevant published measurements. The result is a statistical exclusion value, which indicates how strongly the measurements disfavour that parameter choice.[1] A collection of parameter points forms a grid. Repeating the comparison across this grid maps regions with different levels of exclusion. Figure 2.1 summarises the process from one parameter point to a complete scan.

[1] Here, exclusion value refers to a CL_s-based quantity. Contur can report observed and expected variants. The specific output fields used in this dissertation are introduced in section 3.5.

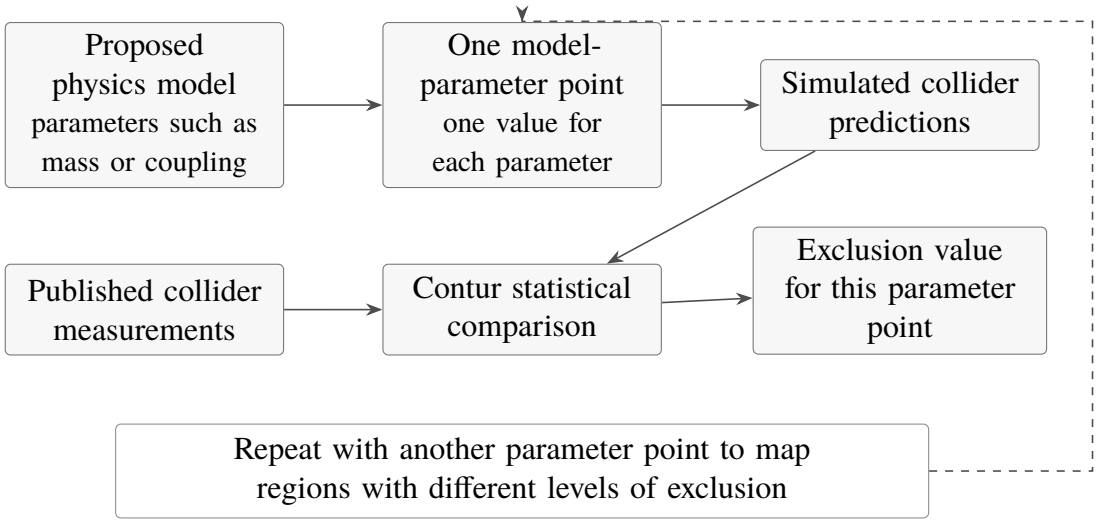

Figure 2.1: Conceptual overview of a Contur parameter scan. One set of model-parameter values is used to produce simulated collider predictions. Contur compares these predictions with published measurements and calculates an exclusion value for that point. Repeating the process for different parameter points maps regions with different levels of exclusion.

The grid result can be displayed as a two-dimensional exclusion map. The example in figure 2.2 makes the discrete grid points, exclusion scale and confidence-level boundaries visible.[2]

The repeated nature of this workflow makes both correctness and efficiency important. A small avoidable cost inside an object created at every point can accumulate across a complete grid. Conversely, an optimisation that changes a statistical result would be unacceptable, even if it substantially reduced runtime. Performance work must therefore include a comparison of the outputs before and after the change.

2.2 Relevant execution path

This section connects the scan-level workflow in figure 2.1 to the software path examined in the performance study. The profiled workflow begins with a Contur grid analysis. Each model-parameter point is referred to in the code as a runpoint. Its inputs and outputs use the YODA histogram and scatter data structures introduced above. Objects defined in the `factories` and `data` packages coordinate the loading and organisation of measurements and predictions. Observable- and likelihood-related code then prepares values for the Contur statistical comparison stage in figure 2.1. Chapter 3 follows this path to explain why `Observable.__getExpected` was invoked at high frequency and why construction of an unused object accumulated substantial cost. The wider package structure is summarised in section 2.3.

[2]The saved tutorial plot was generated from `contur_tutorial/ANALYSIS/contur.map` with `contur-plot contur.map mXm mY1`, where `mXm` and `mY1` are the tutorial names for M_{DM} and $M_{Z'}$. The exact Contur version used to generate the saved plot was not recorded.

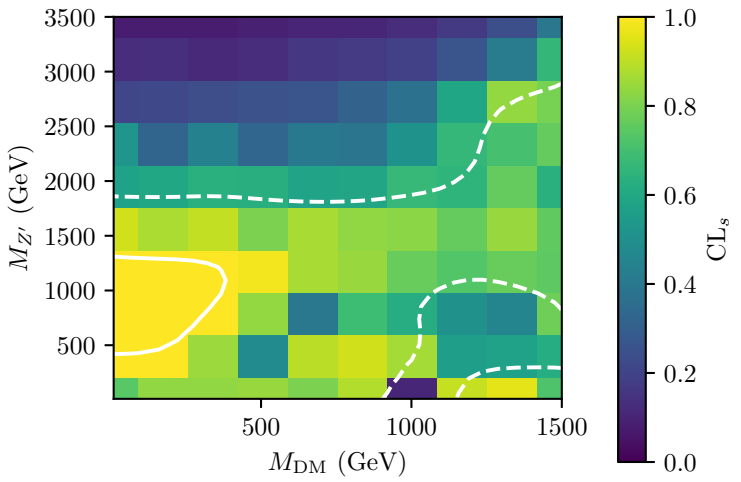

Figure 2.2: Example two-parameter exclusion map from the Contur tutorial. The axes give the dark-matter mass M_{DM} and mediator mass $M_{Z'}$. Colour gives the exclusion value from 0 (not disfavoured) to 1 (strongly disfavoured), while the dashed and solid white contours mark the 68% and 95% confidence-level boundaries. The coarse 10×10 tutorial scan is shown for background explanation, is unrelated to the benchmark grid in chapter 3, and is not a physics result from this project.

2.3 Existing package responsibilities

The official Contur package documentation provides an entry point to the established subpackages and command-line programs [4]. The descriptions below combine that public inventory with the source revisions and branches examined in this project. The list provides the package map used in chapters 4 to 6.

Contur uses a file-based SQLite analysis database to record known Rivet analyses and assign them to orthogonal analysis pools [1, app. H]. In the source revision examined here, the file is named `analyses.db`, and the `static_db` module provides read access to it.

`config` Shared configuration state, paths and logging settings.

`util` General helper functions. At the start of this project, the package also contained some higher-level responsibilities.

`data` Domain objects and data- and covariance-processing functions. At the start of this project it also contained the `static_db` module, which provided access to the static analysis database.

`database` Database access, introduced during this project by relocating the `static_db` module from `data`. The relocation has been merged.

`factories` Construction and coordination of the main Contur worker objects.

`run` Entry-point logic for the command-line programs, calling responsibilities owned by other packages.

`scan` Support for creating, running and processing model-parameter grids.

`plot` Plotting logic and styling for Contur results.

`webpages` Webpage-report generation, introduced during this project by relocating the former `util.rst_utils` module. The relocation has been merged.

`oracle` Iterative selection of additional model-parameter points using a classifier trained on existing Contur results [5].

The list describes current placement rather than claiming that the package boundaries are clean. `static_db` has been moved from `data` into the dedicated `database` package, as recorded in chapter 5. Across the inspected architecture revisions, however, Structured Query Language (SQL) access remains distributed across `database`, `data`, `factories` and `plot`. The `database` package also constructs objects defined in `data`, while some of those data objects call back into `database`. This bidirectional interaction remains the central open problem in chapter 6.

2.4 Architectural concepts used in this study

The dependency analysis uses three related principles. First, package dependencies should not form cycles, because a cycle makes the participating packages difficult to change or release in isolation. Second, dependencies should follow the intended direction between higher-level coordination and lower-level services rather than allowing a nominal foundation package to import its callers. Third, package placement should reflect dependency stability. A package becomes harder to change as more packages depend on it, so a widely used foundation package should contain stable, lower-level responsibilities rather than higher-level workflow or output code that is more likely to change. These ideas correspond to the package and dependency principles described by Martin [6].

The first principle frames the `database`–`data` cycle examined in chapter 6. The second is used to classify the upward and cross-layer paths in table 4.2. The third motivates the responsibility moves recorded in chapter 5.

Figure 2.3 summarises this intended direction. Its Contur-specific application is the target package structure shown later in figure 4.1.

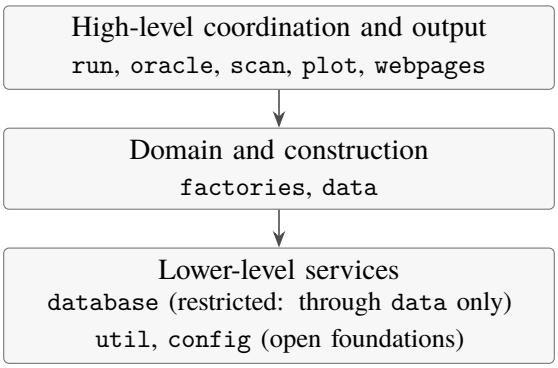

Figure 2.3: Conceptual dependency hierarchy used in this study. Arrows show the intended direction towards lower-level services. The boxes group packages that are separated into more precise levels in figure 4.1. This is a target principle rather than the current package structure.

Output packages such as `plot` and `webpages` belong at the high level because they consume and present lower-level results rather than providing general services to the rest of the package.

Applied to this project, `run` remains a lightweight entry and coordination layer rather than owning webpage formatting, while `util` may depend internally on `config` but not on higher-level Contur packages. Standard-library and third-party dependencies are outside this internal package rule. These principles do not by themselves determine the final boundary between `database` and `data`. Chapter 6 therefore treats that boundary as an open design question and evaluates possible directions without prescribing a final implementation.

2.5 Relationship to previous optimisation work

An earlier MSc dissertation by Sean Bray profiled and optimised Contur. Runtime hotspots were used to select changes, and regression tests checked that the outputs did not change [7]. This previous work provides a useful methodological example for the performance study in chapter 3: record a baseline profile, make a targeted change, and compare the results before and after.

This project has a different scope. It includes one substantial performance change identified by profiling, but most of the ongoing work examines dependencies between packages rather than a series of function-level runtime optimisations. The two parts therefore require different evidence. The performance work uses runtime measurements and output comparisons. The architecture work uses dependency graphs, non-blocking Import Linter checks, focused tests, and analysis of where responsibilities belong.

2.6 Criteria for a useful architectural improvement

The dependency principles in section 2.4 are applied through three criteria.

1. **Clear ownership and dependency direction.** A change should make it easier to identify which package owns a behaviour and to predict which parts of the code it may affect. Dependencies should follow the intended package direction.

2. **Preserved behaviour and focused testability.** A change should not alter behaviour. The affected code should be testable without importing unrelated higher-level workflow code or setting up the whole application, so focused regression tests can check the intended behaviour.

3. **Limited and reviewable scope.** A change should address one responsibility at a time so that its diff and effects can be reviewed clearly. A move-only responsibility relocation has no intended behavioural change and provides a stable base for later work.

These are design criteria used to assess the changes in this dissertation rather than measured long-term maintenance outcomes.

The architecture work is also classified on a scale from local mitigation to automatic protection against regression.

Runtime workaround.
A lazy or function-local import may avoid a runtime import failure, but it does not by itself remove a dependency from a static import graph.

Source clean-up.
Removing an unused or unnecessary import can remove a source dependency without changing where the underlying responsibility belongs.

Responsibility relocation.
A responsibility is moved to the package that should own it. A move-only relocation changes its location without intending to change its behaviour.

Enforceable architectural boundary.
An automated rule blocks a change that violates the intended boundary, so the violation cannot be merged.

This scale is used in chapter 4 to distinguish the non-blocking Import Linter check from an enforceable boundary, and in chapter 5 to describe the completed source clean-up and responsibility relocations.

Chapter 3

Performance Profiling and Optimisation

This chapter explains why `Observable.__getExpected` was invoked at high frequency and why construction of an unused YODA scatter object accumulated substantial cost. It follows the execution path introduced in section 2.2. Section 3.1 establishes a reproducible profiling method, and section 3.2 traces the cost from the full grid workflow to the target method. Sections 3.3 and 3.4 record the rejected and accepted implementation changes. Section 3.5 then checks selected statistical outputs, and section 3.6 summarises the performance result and its limitations.

3.1 Profiling methodology

The performance investigation combined external wall-clock timing with `cProfile`, SnakeViz, `gprof2dot` with Graphviz, and `line_profiler`. External timing supplied the primary elapsed-time comparison. `cProfile` measured the complete grid workflow, SnakeViz and `gprof2dot` presented its call structure, and `line_profiler` inspected selected functions in more detail [8–11]. Their roles and limitations are summarised in table 3.1. Within each paired comparison, the baseline and modified revisions used the same workload and execution mode. Each revision's `src` directory was loaded explicitly through `PYTHONPATH` so that the imported Contur module came from that worktree rather than the package installed in the environment.

3.1.1 Complementary roles of the profiling tools

The tools were not interchangeable. In particular, SnakeViz and `gprof2dot` did not produce independent performance measurements: both presented the same `cProfile` statistics in different visual forms [9, 10]. Their value was in making different questions easier to answer. The workflow therefore progressed from whole-program measurement, through visual exploration of the call structure, to line-level investigation of a selected function.

Table 3.1: Roles, uses and limitations of the performance-analysis tools.

Tool	What it shows	Use in this project	Limitation
`cProfile`	Ranks functions by total and cumulative time.	Found the expected-value construction hotspot.	Adds overhead and gives no line-level detail.
SnakeViz	Shows `cProfile` data as an interactive call hierarchy.	Traced the expensive branch to `Observable`.	Adds no new measurement.
`gprof2dot` with Graphviz	Draws a static caller–callee graph.	Cross-checked the hotspot call path.	Large graphs can become crowded.
`line_profiler`	Measures individual lines in selected functions.	Separated cloning from repeated point updates.	Requires functions to be chosen first.
External timing	Measures elapsed time outside the profiler.	Provided the primary baseline-to-modified benchmark.	Does not locate hotspots.

This division of roles also shaped the investigation. The baseline `cProfile` output was opened in SnakeViz, and branches with high cumulative time were followed down the call hierarchy. As shown in figure 3.1, this led from the grid-processing path through `process_runpoint` and `build_observables` to `Observable.__getExpected`, defined in the production module `contur.factories.test_observable`. Selected levels show approximately 819 s in `process_grid`, 731 s in `process_runpoint` and 730 s in `contur_point.process`. The branch narrows further to 615 s in `build_observables` and 300 s in `Observable.__getExpected`. Its width made it a candidate for closer inspection. The visualisation did not by itself show that the work was unnecessary.

Function-level statistics and `line_profiler` were then applied to the target method. These separated the cost of cloning the reference scatter from the repeated point updates. Manual inspection was still required to determine whether the expensive object was consumed downstream. This combination of whole-program profiling, call-path inspection, line-level measurement and code audit forms a workflow that can be reused when investigating further Contur hotspots.

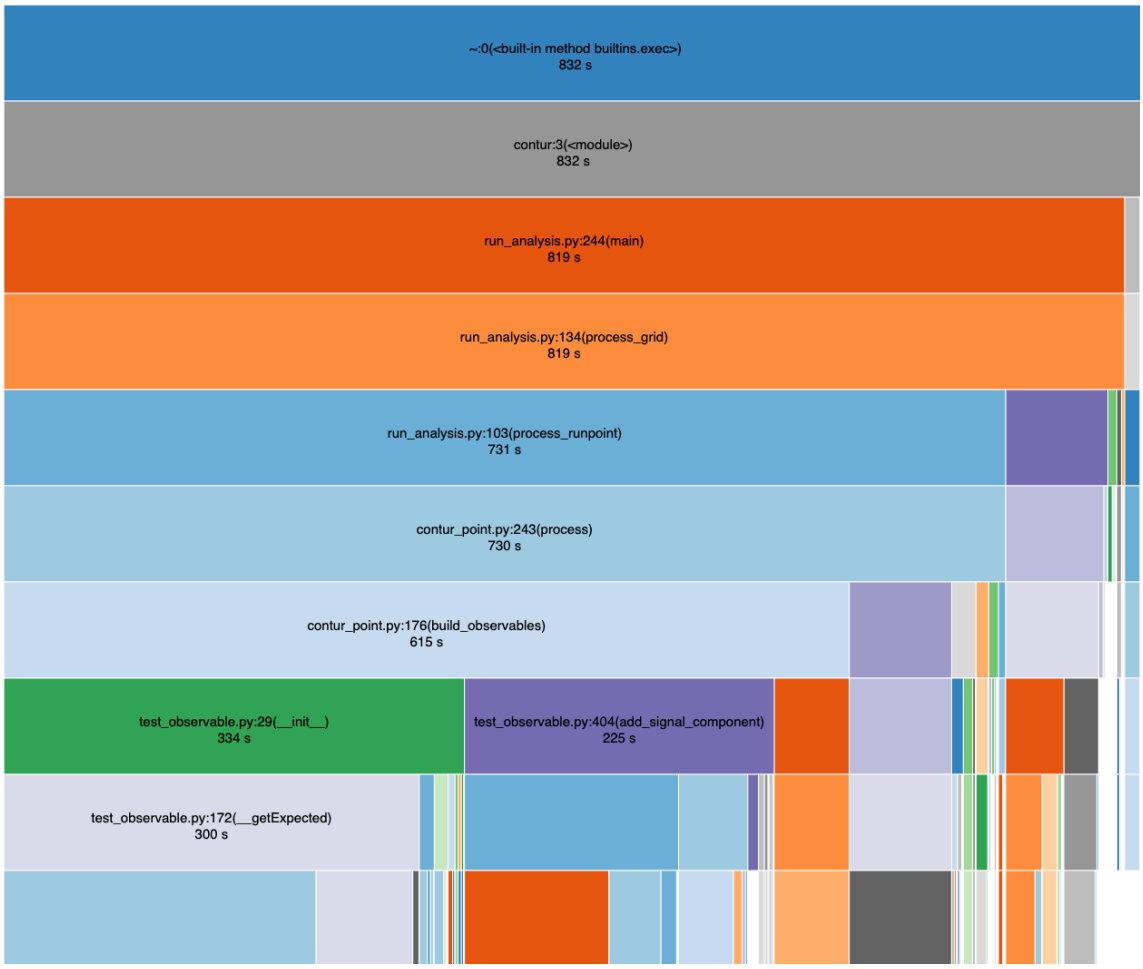

Figure 3.1: SnakeViz icicle view of the controlled baseline profile. Rectangle width represents cumulative time and the displayed times are rounded by SnakeViz. The unprofiled runs remain the primary wall-clock benchmark. The filename `test_observable.py` refers to Contur's production module `contur.factories.test_observable`, which defines `Observable`; it is not a test run. The unlabelled rectangles along the bottom are narrower callees for which SnakeViz does not display text at this scale.

3.1.2 Measurement environment and workload

This subsection defines the source revisions, software environment, workload and execution protocol used for the controlled comparison. Separate detached Git worktrees were prepared at the baseline revision `cb0d0f1866` and optimisation revision `d180115248`.

Explicit source selection proved important. An early apparent post-change full-grid result was effectively unchanged from the baseline because the command had not explicitly selected the edited `src` tree. It was therefore rejected. The valid optimised result was obtained only after adding the worktree's `src` directory to `PYTHONPATH`. This incident is retained as a methodological result: executable names alone are insufficient evidence that the intended Python source revision was measured.

The controlled rerun used an Intel Xeon E3-1270 v6 processor with Python 3.9.12, Contur 3.1.4, Rivet 4.1.3 and YODA 2.1.3. Both worktrees reported this Contur version. The workload comprised a 10×10 grid of 100 model-parameter points. Each point had one compressed YODA file at each of 7, 8 and 13 TeV, giving 300 input files in total.

Before every recorded execution, the 300 input files were read once to warm the file cache. Each revision was then executed three times with `--nomultip`, which disables multiprocessing and runs the workload serially. Serial execution keeps the worker activity in the process observed by `cProfile`. The unprofiled timing runs used the same execution mode so that the baseline and modified revisions were compared consistently, and elapsed time was measured externally. Runs were made in the same session and alternated baseline then modified within each repetition. The resolved Contur module path and full Git revision were recorded automatically. System load was not controlled independently, so the repeated-run spread is reported alongside the mean.

The unprofiled commands had the following form, with both `PYTHONPATH` and the executable path set to the same source tree. The placeholder `<grid_snapshot>` denotes the same fixed directory of 300 input files used for both revisions.

Listing 3.1: Source selection and command shape used for the repeated wall-clock benchmark.

```
export PYTHONPATH=<revision>/src:$PYTHONPATH
python <revision>/bin/contur -g <grid_snapshot> --nomultip
```

3.2 Baseline profile

The controlled baseline profile reported 831.269 s and 1,156,920,804 function calls. Within this profile, `Observable.__getExpected` was called 120,500 times, with 72.202 s of internal time and 300.498 s of cumulative time.

The hotspot was located in two stages. First, the baseline `cProfile` statistics were ranked by cumulative time and the corresponding icicle view was inspected in SnakeViz. Following the widest grid-processing branch led from `process_runpoint`, through `contur_point.process` and `build_observables`, to the construction of an `Observable` and its `__getExpected` method, as shown in figure 3.1. A `gprof2dot` call graph was used to cross-check the caller–callee path. These whole-program views identified an expensive branch, but cumulative time includes callees and does not attribute cost to individual source lines.

The text form of the profile was also inspected with `pstats`. An abbreviated extract is shown in table 3.2. The columns distinguish time spent in the function body (`tottime`) from time spent in the function and all functions called beneath it (`cumtime`). Although functions near the top of the call path necessarily accumulated more cumulative time, sorting by internal time placed `__getExpected` first in the complete 6,217-function profile. Its high call count and its position inside the expensive `Observable` construction branch made it the next line-profiling target.

Table 3.2: Abbreviated `cProfile` output from the controlled baseline run. Times are seconds. The first three rows show the caller path when sorted by cumulative time. The final row is the selected hotspot.

Function	Calls	`tottime`	`cumtime`
`process_runpoint`	100	1.034	731.413
`contur_point.process`	100	1.488	730.378
`build_observables`	100	1.133	614.541
`Observable.__getExpected`	120,500	72.202	300.498

Second, selected methods were inspected with the Python API of `line_profiler`. As shown in listing 3.2, the profiling script instantiated `LineProfiler`, registered seven candidate methods with `add_function`, and wrapped the top-level analysis function. Python's name-mangled attributes were used to register the private methods. This distinction is important: no `@profile` decorators were added to the production source, and `line_profiler` did not select the globally slowest function automatically. Candidate selection could in principle be scripted from the `cProfile` rankings, but this project used manual call-path inspection because cumulative time alone did not determine which functions warranted line-level analysis. The candidate methods were registered only after the full-grid `cProfile` investigation had narrowed the search.

Listing 3.2: Python API driver used to register the selected methods for line-level profiling.

```
import sys
from line_profiler import LineProfiler

from contur.run.arg_utils import get_args
from contur.run.run_analysis import main
from contur.factories.test_observable import Observable

lp = LineProfiler()
lp.add_function(Observable.__init__)
lp.add_function(Observable._Observable__getExpected)
lp.add_function(Observable._Observable__getData)
lp.add_function(Observable._Observable__getThy)
lp.add_function(Observable._Observable__getAux)
lp.add_function(Observable.add_signal_component)
lp.add_function(Observable.build_likelihood)

args = get_args(sys.argv[1:], "analysis")
```

```
18
19 try:
20     lp_wrapper = lp(main)
21     lp_wrapper(args)
22 finally:
23     lp.print_stats()
```

The exploratory line profile was run interactively in the UCL HEP computing environment using one compressed YODA runpoint. Its complete logs and driver script were retained, but the run was not part of the controlled full-grid benchmark. It is therefore used for source-line attribution only. The output used a timer unit of 10^{-9} seconds and reported one row per source line, including the execution count (`Hits`), raw time, time per hit and percentage of time within the selected method.

The recovered baseline output is abbreviated in table 3.3. `__getExpected` was entered 958 times. Of these, 64 invocations returned before constructing an expected scatter, leaving 894 executions of the clone. Of the method's total 10.3573 s, 24.2% was attributed to cloning and 72.1% to the per-point update. Together these two operations accounted for 96.3% of its line-profiled time.

Table 3.3: Abbreviated recovered `line_profiler` output for `Observable.__getExpected` on one runpoint. Times are seconds. Percentages are shares within the selected method, not percentages of whole-program runtime.

Source operation	Hits	Time	Within method
Clone the reference scatter	894	2.5018	24.2%
Evaluate the point-iteration loop	10,718	0.3487	3.4%
Copy the theory value with `setY`	9,824	7.4698	72.1%

The differing hit counts reflect the nesting of the code: the clone is executed at most once per non-returning invocation, whereas the loop condition and body execute once per scatter point. The line profile therefore separated the cost of creating the YODA object from the repeated point mutations. It identified where time was spent, but it could not establish whether the constructed object was needed.

The same single-runpoint output reported 18.850 s within `__getData`, mainly in copies of correlation, error and uncorrelated-error data. This exceeded the selected `__getExpected` total in that exploratory workload, illustrating why these line-profile totals were not used to rank whole-program hotspots. Unlike the constructed expected scatter, the values produced by `__ge tData` were consumed by subsequent calculations.

The inspected source lines cloned a YODA reference object and copied theory values into the resulting scatter:

Listing 3.3: Removed construction of the unused expected scatter.

```
1 self._expected = self._ref.clone()
2 for i in range(len(self._expected.points())):
3     self._expected.points()[i].setY(self._thy.points()[i].y())
```

Subsequent code audit indicated that downstream likelihood calculations used `expected_values` rather than the constructed `self._expected` scatter. The observed cost was therefore a candidate for removal rather than merely faster implementation.

3.3 Exploratory caching attempt

The first attempted change stored the expected and theory point lists in local variables named `exp_points` and `thy_points` before the loop. This avoided repeated method calls but left the expected-scatter construction in place. A back-to-back `cProfile` comparison did not show a clear improvement. Subsequent inspection then showed that `self._expected` was not read after construction. The caching change was therefore not pursued, and the investigation moved from making the loop cheaper to removing the unused construction. The original caching-comparison profiles were not retained, so this attempt is not included in the controlled quantitative result below.

3.4 Removal of redundant work

The final change removed construction and mutation of `self._expected` while preserving the values consumed by the statistical path. The implementation was subsequently merged upstream.

On the same historical single-runpoint line-profile workload, the time recorded within `__getExpected` fell from 10.3573 s to 0.0168 s, and the clone and `setY` rows disappeared. Both processes exited successfully, and the compared output values were identical. The corresponding externally observed elapsed times were 8:26.14 and 8:19.43, however, so this instrumented single-runpoint pair is evidence for the removal mechanism rather than an end-to-end speed-up claim. The repeated unprofiled full-grid experiment below remains the primary performance comparison.

Table 3.4: Repeated unprofiled wall-clock benchmark before and after removal of the unused scatter construction. Values are seconds. The uncertainty shown is the sample standard deviation across three runs.

Revision	Mean ± SD	Median	Range
Baseline	601.75 ± 4.18	602.57	597.22–605.46
Modified	386.77 ± 3.19	387.14	383.41–389.75

The mean elapsed time fell by 214.98 s, corresponding to a 35.73 % reduction and a 1.56-fold speed-up for this workload.

The profile supports the proposed mechanism. The total profiled time fell by 297.175 s, while the cumulative time attributed to `__getExpected` fell by 299.844 s. The corresponding reduction in `Observable.__init__` should not be added to this value because `__getExpected` is called within the constructor and cumulative times therefore overlap. The close agreement between the local and total reductions is consistent with the work having been removed rather

Table 3.5: Diagnostic `cProfile` comparison. One profiled run was performed per revision. These times include profiler overhead and are not the primary wall-clock benchmark.

Metric	Baseline	Modified	Change
Profile total (s)	831.269	534.094	−35.75%
Total function calls	1,156,920,804	736,177,812	−36.37%
`__getExpected` cumulative time (s)	300.498	0.654	−99.78%
`Observable.__init__` cumulative time (s)	334.119	34.209	−89.76%

than shifted elsewhere in the measured path. However, this does not prove that every other cost stayed unchanged.

The profiled reduction of 35.75 % is also close to the unprofiled mean reduction of 35.73 %. This agreement supports the same interpretation, although the single profiled run per revision is diagnostic rather than an independent repeated benchmark. These values should not be generalised to every machine, input grid or Contur workflow without further measurement.

3.5 Behavioural validation

Each completed run produced 300 ordered core-output rows: `SMBG`, `EXP` and `HLEXP` for each of 100 runpoints. `SMBG` uses the available Standard Model prediction as the background. `EXP` is the expected limit obtained by moving the data central value to that prediction, while `HLEXP` applies the expected-limit calculation with uncertainties scaled for projected High-Luminosity Large Hadron Collider data. The baseline and modified comma-separated value (CSV) extracts matched exactly in all three unprofiled pairs and in the additional profiled pair. This provides full-workload evidence for the extracted statistical outputs, but it is not a proof of equivalence for every database field, generated artefact or possible configuration.

3.6 Performance conclusions

The performance strand produced one substantial, behaviourally checked optimisation and a profiling workflow that can be reused for later investigations. After the redundant construction was removed, the retained profiles did not indicate another comparably clear source of avoidable cost that could be addressed without wider structural changes. Further optimisation was therefore not pursued through speculative function-level changes. Instead, the project turned to the package structure that would need to support broader changes safely. The following chapter begins this second strand by examining Contur's dependencies and responsibility boundaries.

Chapter 4

Dependency and Architecture Analysis

This chapter begins the second strand of the dissertation: analysis of package dependencies and responsibility boundaries.

4.1 Analysis methods

The architecture investigation combined static tools with manual code audit rather than treating one tool's output as a complete diagnosis. Pylint `cyclic-import` reports provided textual paths through detected cycles. Pyreverse exported a broader module representation using Mermaid, a text-based syntax for describing diagrams that records the module relationships as text rather than as a rendered image. The JavaScript Object Notation (JSON) output from `pydeps --show-deps` listed each module's imports and `imported_by` relationships, providing a separate cross-check [12–14]. These tools showed which modules imported each other, but they could not decide whether the dependencies were appropriate. This required manual inspection of the code and package responsibilities.

The retained pydeps diagram was generated as a focused module-neighbourhood view centred on `grid_tools`. It was used to inspect the `scan–run` module-load-time cycle involving `grid_tools` and `run_batch_submit`. The `--max-bacon 1` setting limited the diagram to the target module's one-hop neighbourhood, so it was not used to count cycles across the whole package. The manual audit then checked import locations, callers and module responsibilities. The proposed package boundaries were later written as repeatable Import Linter checks [15].

Static checking is important because it analyses imports without relying on a particular runtime import order. Moving an import inside a function can defer execution and avoid an immediate failure, but the architectural dependency still exists in the source. Such a change is not counted here as resolution of the underlying boundary.

4.2 Baseline package dependencies

The existing structure was first recorded without judging which directions were desirable. A Python script used abstract syntax tree (AST) parsing through the standard-library `ast` module to examine every file under `src/contur` at revision `cb0d0f1866`. One imported target pack-

age on an `import` or `from...import...` statement was counted as one direct import. The count includes module-scope imports, function-local imports and imports inside `TYPE_CHECKING` blocks. Imports within the same top-level Contur package and third-party imports were excluded. The script, per-import output and matrices are retained in the project evidence directory. The baseline run parsed 64 Python files, and no source file failed to parse.

Table 4.1: Cross-package direct imports at the baseline revision. Rows are importing packages and columns are imported packages. The totals contain all 167 counted imports and do not classify any direction as desirable or undesirable.

	Imported package								
Importing package	`run`	`plot`	`scan`	`oracle`	`factories`	`data`	`util`	`config`	Outgoing
`run`		7	10	0	10	14	17	18	76
`plot`	0		0	0	3	3	4	5	15
`scan`	1	0		0	0	1	7	7	16
`oracle`	0	0	0		0	0	0	1	1
`factories`	0	3	0	0		10	6	11	30
`data`	0	1	2	0	2		4	8	17
`util`	1	0	0	0	1	3		7	12
`config`	0	0	0	0	0	0	0		0
Incoming total	2	11	12	0	16	31	38	57	167

The matrix gives a package-level view of the starting design. `config` and `util` received the most incoming cross-package imports, with 57 and 38 respectively, while `run` had the largest outgoing total at 76. This is consistent with configuration and general helpers being widely reused and with `run` coordinating work owned by several other packages. The matrix alone does not determine whether any individual direction is appropriate. It also does not expose responsibility questions within individual modules. The module-level audit of `util` in section 4.4.1 examines those questions separately.

4.3 Target dependency structure

The architectural analysis produced the working target shown in figure 4.1. It retains `util` and `config` as open foundation packages, places webpage generation at a higher level, and restricts direct access to `database` to the `data` package. Both `data` and `database` may use the general helpers in `util`. Current reverse dependencies and other violations are intentionally omitted from the diagram. Observed import counts are kept in tables 4.1 and 4.2 rather than written on these arrows, because the counts describe recorded source revisions whereas the diagram describes permitted directions.

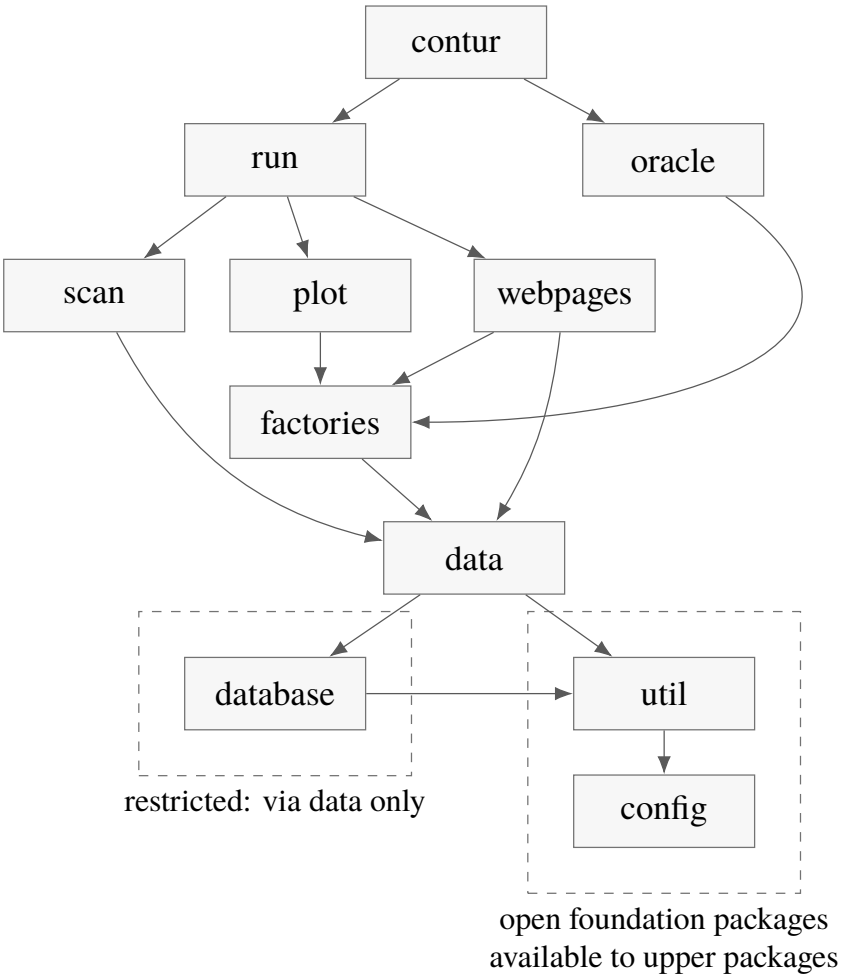

Figure 4.1: Target package dependency structure. An arrow from *A* to *B* indicates that package *A* may import package *B*. `util` and `config` are open foundation packages available to upper packages. `database` is a restricted service that only `data` may access, although `database` may use `util`. This is the intended target rather than the current state. The baseline structure is recorded in table 4.1, target conflicts are compared in table 4.2, and the remaining webpage database access is discussed in section 5.5.3.

The target permits `data` to import `database`, but not the reverse. The current dependency from `database` to `data` therefore conflicts with the restriction and is absent from the target diagram. It remains part of the cycle discussed in chapter 6.

4.4 Dependencies that conflict with the target

The baseline column in table 4.2 is the original GitLab continuous integration (CI) snapshot from the `importlinter-config` branch at commit `c88cd32e61`. The branch was based on `cb0d0f1866`; its four commits added and simplified the Import Linter configuration without changing files under `src/contur`. At this point, `static_db` still belonged to `data` and the dedicated

`database` package did not exist. The CI job used Python 3.9, installed `import-linter>=2.0`, and executed `lint-imports --verbose`. It reported 14 direct violations across eight package directions. The exact resolved Import Linter version was not retained in the available job record. The run is GitLab job 15420374888 in pipeline 2688653739.

The AST census was then applied to the baseline and post-change revision `cf6ca9b157` using the same counting rule. Applying the target directions in figure 4.1 to the baseline reproduced the 14 imports and eight directions from the CI snapshot exactly. This agreement checks that the census is comparable with the recorded Import Linter result. The post-change source was the revision recorded locally as `origin/main` at the time of the census. The baseline and post-change runs parsed 64 and 66 Python files respectively, with no parse failure. This architecture revision is separate from the optimisation revision used for the performance comparison in section 3.1.2.

Table 4.2: Cross-package direct imports that conflict with the target dependency structure at the baseline and post-change revisions. Negative changes record removed conflicts. Positive changes record conflicts made visible by the new package structure.

Package direction	Baseline	Post-change	Change
`util` → `factories`	1	0	−1
`util` → `data`	3	0	−3
`util` → `run`	1	0	−1
`data` → `scan`	2	2	0
`data` → `factories`	2	2	0
`data` → `plot`	1	1	0
`scan` → `run`	1	1	0
`factories` → `plot`	3	3	0
`database` → `data`	0	1	+1
`run` → `database`	0	7	+7
`plot` → `database`	0	2	+2
`scan` → `database`	0	1	+1
`webpages` → `database`	0	1	+1
`factories` → `database`	0	5	+5
Total	14	26	+12

Against the target, the five original upward imports from `util` were removed. The nine other original conflicts were unchanged. Moving `static_db` into the new `database` package made sixteen direct accesses from upper packages visible against the target restriction that only `data` may access `database`. The reverse `database`–`data` direction added one further conflict. The increase from 14 to 26 therefore records unfinished database-boundary work rather than implying that every architectural change made the design worse. The complete cross-package count changed only from 167 to 170 because most newly visible conflicts were not new import statements. Imports from upper packages that previously targeted `data.static_db` were already cross-package imports and were reclassified as restricted `database` access after the move. The original 14 direct imports are listed in appendix A.

For context, an earlier like-for-like Pylint comparison reported five `cyclic-import` warn-

ings (R0401) in each of two historical snapshots. Each warning recorded one path through modules participating in a detected import cycle. One source file had to be excluded, so that result is retained only as a qualitative cross-check. The recorded Import Linter CI snapshot covers the package source tree at its stated revision and remains the independent quantitative check on the baseline census.

4.4.1 Utility audit

A key working principle emerged from this analysis: a general low-level utility package should not need to import high-level packages containing domain, factory or run logic. This principle motivated a function-by-function audit of `contur.util.utils`. Functions were classified as general helpers, helpers coupled to higher-level objects, candidates for relocation, or dead-code candidates requiring a separate review. The caller search found no in-tree uses of `insert_line_break`, `Plot` or `match_analysis_objects`. External use through Contur's Python interface could not be excluded, so none of these symbols was claimed as dead code or removed. The audit therefore did not confirm any unused function. A separate clean-up did confirm and remove an unused import from `file_readers.py`, as described in section 5.1.

The file-level audit found that most functions already satisfied the proposed dependency rule. General helpers such as `cleanupCommas`, `get_numcores`, `analysis_select` and `histo_select` could remain because they did not import higher-level Contur packages. Following responsibility review, `newlogspace` was moved to `scan`, its only owning workflow, and focused tests were added. Other symbols were retained or marked for separate cohesion discussion rather than being counted as dependency violations.

A separate package-level review identified `util.rst_utils` as a clearer responsibility problem. Unlike the individual helpers in `utils.py`, this module generated webpage reports and imported both database and factory concerns. The merged relocation described in section 5.5 therefore addresses a misplaced high-level module rather than splitting `util` into multiple utility tiers.

4.5 Import Linter contracts

Import Linter contracts are used as repeatable checks against the current codebase. At this stage they represent testable hypotheses about module boundaries rather than an accepted final layering, so they can be revised as responsibilities are clarified.

After the initial configuration was simplified and the new `database` package was added to the proposed layers, GitLab CI reported 12 direct violations across eight directions at commit `a1985475be` on the `importlinter-config` branch. The job used Python 3.9, Import Linter 2.5.2 and Grimp 3.13, and executed `lint-imports --verbose`. This run included the corrected `run_mkthy.py`. The earlier Pylint comparison in section 4.4 had excluded this file because its recorded revision contained unresolved Git conflict markers. Commit `32252f1f67` removed those markers before the later CI run. Its values are retained as an intermediate contract run, but they are not used as the post-change column in table 4.2 because the contract did not yet encode the complete target.

The intermediate violations included one `util` → `database` import from `util.rst_utils`. The merged webpage relocation moved this module to `webpages.report`. The post-change census therefore records the remaining direct access as `webpages` → `database`, as discussed in section 5.5.3.

The contract and its dedicated GitLab CI job are recorded in draft merge request `!652`. The job returned exit status 1 because the contract was broken, while `allow_failure: true` made this a pipeline warning rather than a merge blocker. The accompanying `pytest` job passed. The contract is therefore an observation mechanism, not an enforced boundary. The merge request is being kept as a draft branch while the circular dependencies are addressed. It will be updated and rerun against the later revision before it is merged into `main`. Once the agreed contract passes, removing `allow_failure: true` would cause a newly disallowed import to fail the pipeline instead of appearing only as a warning. This provides automatic regression protection by requiring the code or the contract to be corrected before the pipeline can pass. The check should become blocking only after the current known violations have been resolved, so that the existing broken baseline does not obstruct unrelated changes. Only then can the contract be described as an enforced boundary.

The executable contract used for the intermediate CI run is shown in listing 4.1. It records the configuration at commit `a1985475be` and only partially encodes the target. It predates the merged `webpages` package. The linear order in the listing is the provisional contract order, not a representation of the complete target graph. A general layered contract permits upper layers to import any lower layer. Placing `database` below `data` therefore permits direct access from all upper layers and does not enforce the target rule that such access must pass through `data`.

Listing 4.1: Current provisional Import Linter contract used for the recorded CI snapshot.

```
[tool.importlinter]
root_package = "contur"

[[tool.importlinter.contracts]]
id = "proposed-layers"
name = "proposed layered architecture"
type = "layers"
layers = [
    "contur.run",
    "contur.plot | contur.scan | contur.oracle",
    "contur.factories",
    "contur.data",
    "contur.database",
    "contur.util",
    "contur.config",
]
```

This listing is retained as the reproducible intermediate contract rather than presented as the final configuration. Draft merge request `!652` will remain separate from `main` until the circular dependencies have been resolved and the contract has been revised and rerun.

Chapter 5

Implemented Architectural Improvements

This chapter records the architectural changes made during the project. It distinguishes merged responsibility moves from the database-boundary work that remains on a feature branch. Most changes were kept small so that they could be reviewed without also reviewing the larger database redesign.

Before moving these functions and modules, focused tests were added for file paths, analysis and histogram selection, and rules for including or excluding results. These tests check that existing behaviour does not change when code is moved. However, passing these tests does not mean that the target architecture has been completed.

The classification in table 5.1 follows the scale defined in section 2.6. The first change is source clean-up. The other four are responsibility relocations with no intended behavioural change.

Table 5.1: Status and architectural effect of the changes discussed in this chapter.

Change	Classification	Recorded status	Architectural effect
Clean imports and names in `util.file_readers` and `util.rst_utils`	Source clean-up	!653. Merged. Pipeline passed.	Removed an unused upward import and made later dependency analysis clearer.
Move `find_thy_predictions` from `util.utils` to `factories.yoda_factories`	Responsibility relocation	!654. Merged. Pipeline passed.	Removed theory-data lookup from the general utility module.
Move `static_db` from `data` to the new `database` package	Responsibility relocation	!657. Merged. Pipeline passed.	Gave static database access a named package while leaving the database–data cycle visible.
Move `newlogspace` from `util.utils` to `scan.scanning_functions`	Responsibility relocation	!674. Merged. Pipeline passed.	Placed a scan-only helper with the workflow that uses it.
Move `rst_utils` from `util` to the new `webpages` package	Responsibility relocation	!678. Merged. Pipeline passed.	Removed a high-level responsibility from `util`, but the `webpages` → `database` dependency remains.

5.1 Utility import clean-up

The first clean-up concerned imports and names in `file_readers.py` and `rst_utils.py`. In particular, `file_readers.py` imported static database code even though it did not use that import. This made the utility package appear more dependent on higher-level code than the implementation required.

Merge request !653 removed the unused import, made the Herwig helper import explicit, and corrected undefined names. The change was merged and its pipeline passed.

This removed one of the three direct `util`–`data` imports recorded in the original Import Linter snapshot listed in appendix A. It did not redesign package ownership, but it reduced noise before the responsibility moves below.

5.2 Relocation of theory-prediction lookup to `factories`

The function `find_thy_predictions` was previously located in `contur.util.utils`. The function queries Standard Model theory predictions and therefore used data-layer behaviour that did not belong in a general utility module.

Merge request !654 moved the function to `contur.factories.yoda_factories` and updated its caller. The change was merged and its pipeline passed.

The source change removed the direct `util.utils`–`data` import used by this function. It also placed the lookup beside the YODA construction code that consumes its result. This is a specific import-path improvement rather than evidence that every `util` dependency in the baseline column of table 4.2 has been removed.

5.3 Relocation of static database access to `database`

Static access to `analyses.db` was previously implemented in `contur.data.static_db`. Its main work is loading, caching and querying database content, not defining the domain objects themselves. Keeping it in `data` mixed these two responsibilities.

Merge request !657 moved the module to `contur.database.static_db`. It created the `database` package and updated imports, package exports, documentation and tests. The change was deliberately kept as a relocation rather than a rewrite of the queries. It was merged and its pipeline passed.

The move gave static analysis-database access a clear package name, but it did not move all SQL access into that package. It also left `static_db` constructing objects defined in `data`. The resulting database–data boundary remains open and is examined in chapter 6.

5.4 Relocation of logarithmic scan spacing to `scan`

The utility audit found that `newlogspace` had no upward package dependency, but it was used only when constructing scan axes. This was a question of ownership and code organisation, not one of the conflicts in the baseline column of table 4.2.

Merge request !674 moved the function from `contur.util.utils` to `contur.scan.scanning_functions`. Two focused tests check the requested logarithmic bounds and the use of logarithmic spacing during scan-axis construction. The merge request was merged and its pipeline passed.

The change places a scan-only helper beside its caller and keeps its behaviour covered by focused tests. It improves cohesion, but it should not be counted as a reduction in the Import Linter violation total.

5.5 Relocation of webpage reporting to `webpages`

5.5.1 Responsibility problem

The former `contur.util.rst_utils` module generates webpage reports in reStructuredText and starts the related Standard Model histogram plots. Its placement made the low-level `util` package import `database` at module scope and `factories.yoda_factories` inside a function. Moving the second import inside the function delayed it at runtime, but the source dependency still remained.

5.5.2 Implemented change

Merge request !678 renamed `util.rst_utils` as `webpages.report` inside a new high-level `contur.webpages` package. The `run` entry point now calls this package. The change also restored the YODA factory import to normal module scope, as shown in listing 5.1. The merge request was merged and its pipeline passed.

Listing 5.1: Representative imports before and after moving the webpage-reporting module.

```
# Before: contur/util/rst_utils.py
import contur.database.static_db as cdb

def write_sm_file(ana,out_dir,text_string):
    import contur.factories.yoda_factories as yf

# After: contur/webpages/report.py
import contur.database.static_db as cdb
import contur.factories.yoda_factories as yf
```

The change added two focused tests. One checks that the report functions are available from the new package. The other checks that `run_init` calls the webpage generator. These tests cover the new interface and call path, but they do not yet provide a complete comparison of the generated webpages.

The post-change census in table 4.2 records zero remaining `util` imports into `factories`, `data` or `run`. It also records the remaining `webpages–database` import. The relocation therefore removed the reporting module from `util` without completing the restricted database boundary. A fresh Import Linter run is still required before the draft contract can be treated as enforced.

5.5.3 Remaining work

Although the relocation has been merged, `webpages.report` still imports `database.static_db` directly. This is inconsistent with the target restriction in figure 4.1, under which only `data` accesses `database`. The dependency must therefore be removed or mediated as part of the remaining database-boundary work. The draft Import Linter contract must then be revised and rerun before a post-change Import Linter violation count is reported. The source census in table 4.2 already records the dependencies at the stated post-change revision, but it is not a rerun of the draft contract. A complete comparison of generated webpages also remains outside the two focused interface and call-path tests described above.

Chapter 6

Ongoing Database Boundary Redesign

6.1 Problem exposed by the package move

The `analyses.db` file is the SQLite analysis database introduced in section 2.3. During normal use, it provides the known Rivet analyses, their pool assignments and related metadata. The module `database.static_db` provides the main read path used here.

Moving `static_db` into a dedicated package clarified its ownership but exposed a two-way dependency between the module `database.static_db` and the module `data.data_objects`. This is not only an import cycle. The `database.static_db` module combines reading rows with constructing domain objects, while methods on one of those objects call back into the database to load related information.

The five relevant classes are defined by the `data.data_objects` module.

When `static_db.init_dbs()` first reads `analyses.db`, it constructs `Beam`, `Experiment`, `Pool`, `Analysis` and `SMPrediction` objects. The database package therefore depends on the constructors and representation of objects in `data`.

The dependency also runs in the other direction. Three methods on `Analysis` call `database.static_db` to obtain related objects, as shown in table 6.1.

Table 6.1: Calls from `Analysis` back into `database.static_db`.

`Analysis` method	Call into `static_db`	Result
`get_pool()`	`get_pool(poolid=self.poolid)`	Matching `Pool` object
`sm()`	`get_sm_theory(self.name)`	List of `SMPrediction` objects, or `None`
`hasPrediction()`	`get_sm_theory(self.name)`	Boolean indicating whether predictions exist

The last two methods use the same in-memory lookup after the database has been initialised. This shows that both methods depend on the database service, but it is not evidence that they execute the same SQL query twice.

Together, the two directions make the packages difficult to change separately. Changes to

the domain-object constructors can require changes in `static_db`, while the three relationship methods on `Analysis` depend on objects supplied by the database service. Reading records, constructing objects and loading their relationships are therefore spread across both packages. Moving the module alone did not resolve this boundary, and the redesign must decide where each responsibility should sit.

The function-local import from `Analysis.toHTML()` into plotting code is a separate cross-layer issue included in table 4.2. It is not part of the database–data problem examined here.

6.2 Wider SQL ownership

The audit also found database responsibilities outside `static_db`. Database-building and result-persistence logic remains in `data`. Factory classes execute direct SQL when reconstructing model points, while plotting code reads information directly from result databases. These findings suggest that the static analysis database should be treated as the first bounded case rather than assuming that one module move completes the database architecture.

The audit also distinguishes two physical database responsibilities. The analysis database, `analyses.db`, is static during normal Contur use and is read by `static_db`. Its construction is currently implemented separately. Results and model point databases are read and written during scan processing through code including `data_access_db` and factory-level SQL. The provisional direction is to keep these read-only analysis and read–write results responsibilities in separate modules within `database`, rather than treating all SQL as one interface.

6.3 Candidate responsibility split

One candidate design separates the work into three stages:

1. database functions execute queries and return low-level records.
2. a data access object, repository, loader or mapping component assembles and links domain objects.
3. higher-level callers use the resulting `Analysis`, `Pool` and related objects without issuing SQL directly.

The Repository pattern is one established example of separating domain access from persistence details [16]. It is only one candidate in the broader family being considered here. The project has not selected Repository over a data access object, direct mapping tool or another design. Any of these approaches could remove the need for the database layer to construct higher-level data objects. The existing behaviour of methods such as `Analysis.get_pool()`, `Analysis.sm()` and `Analysis.hasPrediction()` may need to be preserved, and the choice between eager loading, lazy loading and a repository or service requires maintainer agreement.

6.4 Current implementation and evaluation status

Before changing the database boundary, merge request !681 added tests that characterise the existing object-identity behaviour. The merge request was merged and its pipeline passed. The tests show that repeated database lookups reuse the cached `Analysis`, `Pool`, `Beam` and `Experiment` instances. They also show that database, `Analysis` and theory-prediction lookup paths share the same mutable `SMPrediction` instance, and that `obsFinder()` returns the cached `Analysis`. A later implementation must preserve this behaviour unless an intentional interface change is separately agreed and tested.

Open merge request !685 implements the first stage of the candidate split on a feature branch. It introduces immutable low-level records for beams, experiments, pools, analyses and Standard Model predictions, together with a reader that returns these records without constructing Contur domain objects. Focused tests cover record values and field order, immutability, connection closure, the absence of domain-object construction in the low-level reader, and preservation of record-backed values through the existing initialisation path.

This is branch-level progress rather than a completed database boundary. In the current merge request, `static_db.init_dbs()` still constructs the domain objects, and the reverse calls from `Analysis` into `database.static_db` remain. The change therefore creates a testable query-to-record seam but does not yet remove the database–data cycle. Final evaluation also requires the draft Import Linter contract to be revised and rerun. The separate post-change census in table 4.2 records the unresolved source dependencies but does not make the contract enforceable.

Chapter 7

Evaluation and Discussion

7.1 Evaluation framework

The two strands of the project require different forms of evidence. Performance changes are evaluated through controlled profiles recorded before and after the change, together with output comparison. Architectural changes are evaluated through responsibility analysis, dependency violations, focused tests, reviewable change scope and compatibility with existing workflows. Runtime improvement alone cannot show that a package boundary is better, while a cleaner dependency graph cannot show that a numerical optimisation preserved scientific behaviour.

7.2 Performance result interpretation

The removal of an unused YODA scatter object constructed and updated point by point in `Observable.__getExpected` is the strongest completed quantitative result. Across three unprofiled runs, mean elapsed time fell by 35.73 %. The removed work included cloning the reference scatter and copying theory values into its points. The low within-case variation is small relative to this difference, while the separate diagnostic profile attributes the largest local reduction to `Observable.__getExpected`. This supports removal of those repeated object and point operations rather than displacement of the same work elsewhere in the measured call path.

Nevertheless, the benchmark covers one serial, warm-cache workload on a single cluster node, and only three elapsed-time observations were collected for each revision. The 35.73 % reduction should therefore be treated as an observed result for this setting, not a universal speed-up guarantee. Exact agreement across 300 extracted rows in each of four paired comparisons strengthens the behavioural evidence, but broader automated regression coverage would still be needed to cover other database fields, artefacts and configurations.

7.3 Architectural progress

The architectural strand has produced a different kind of progress from the performance strand. It does not yet have a quantitative result showing that the target architecture has been achieved.

Instead, it has produced small changes whose value lies in placing responsibilities more clearly. The merged changes moved a domain-specific query and a scan-specific numerical helper out of `util`, and separated static database access into its own package. This shows that useful changes can be reviewed and merged without waiting for the complete redesign.

The method used for these changes is also important. The database change moved `static_db` without rewriting its query behaviour. This small relocation made the remaining database–data coupling easier to identify and review before a larger redesign was attempted. More generally, keeping each relocation small enough for a focused review allowed its ownership and test evidence to be assessed without combining it with the unresolved database redesign. It also allowed chapter 6 to describe that coupling at method level. The merged `webpages` relocation follows the same approach. It moves report generation and its upward dependencies out of `util`, while leaving the direct `webpages.report–database.static_db` dependency visible for further work.

The merged object-identity tests provide a behavioural constraint for the next database change. The open low-level-records merge request then introduces a query-to-record seam without yet claiming that the database–data cycle has been removed. This sequence makes the preserved behaviour and the unfinished boundary explicit.

A further documentation improvement would be to add package-level docstrings that state the responsibility and permitted dependencies of each package. This has not yet been implemented. Together with the Import Linter contract, these docstrings could record the intended design in both readable documentation and an automated check.

The evidence for the two strands is not yet equally complete. The performance change has a completed benchmark comparing measurements recorded before and after the change. The architectural work is supported by responsibility analysis, a small number of merged changes, a recorded Import Linter snapshot and the like-for-like AST census in table 4.2. That census reports 14 conflicts at the baseline revision and 26 at the post-change revision. The increase does not show that the relocations made the design worse. Moving `static_db` into `database` made existing access from upper packages visible against the target restriction, while the database–data boundary remains unfinished. The evidence therefore shows clearer responsibility placement and identifies the remaining conflicts quantitatively, but it does not show that the target layering has been achieved.

7.4 Threats to validity

7.4.1 Performance limitations

The principal performance threats are environmental load, the small number of repetitions and the use of one recorded grid workload. The repeated elapsed-time comparison avoids profiler overhead, while the separate profile is used only for attribution. System load was not controlled independently, and the accepted runs explicitly selected the intended source tree. The 35.73 % reduction should therefore be read as a result for this serial, warm-cache workload on one node, not as a general guarantee.

A stronger evaluation would use more repetitions and report a confidence interval for the

mean difference. It would repeat the benchmark for several grid sizes and on more than one node, while recording or controlling competing system load. A stable scheduled benchmark could also track the workload across later revisions and reveal performance regressions. Such monitoring would need a controlled runner because an ordinary shared CI runner can itself introduce variable load.

7.4.2 Architecture limitations

The original count in table 4.2 comes from a recorded Import Linter snapshot and was reproduced exactly by the AST census. The post-change column comes from the same AST counting method at the stated later revision, not from a final Import Linter CI run. The source-level direction counts can therefore be compared. They increase from 14 to 26 because the new `database` package makes direct access to `static_db` from upper packages, and the reverse dependency into `data`, visible as conflicts with the target. This measures the remaining boundary work rather than a completed reduction in violations. The draft contract has not yet been revised to encode the complete target. It also runs in CI with `allow_failure: true`, so the job does not currently block a merge when it finds a disallowed import.

The `webpages` relocation has been merged, but it still contains a direct `webpages.report–database.static_db` dependency. The relocation is therefore a completed responsibility move, not evidence that the target dependency boundary has been achieved. The low-level-record implementation also remains on an open branch. Responsibility placement involves judgement from the maintainers and the conventions of the project. Finally, static analysis may not find imports created dynamically at runtime. In the inspected Contur source, runtime imports load user-provided plotting functions or grids and selected standard-library or third-party modules. They were not used to select internal Contur packages, so they are a limited concern for the package-boundary census reported here. A future check could record imports with a runtime import hook while representative commands are executed and compare them with the static census. A layered contract still cannot decide whether every allowed dependency represents an appropriate responsibility.

These limitations do not invalidate the changes already merged. They limit the current architectural conclusion to responsibility analysis, clearer package placement and provisional static checks rather than an enforced or fully quantified redesign.

Chapter 8

Conclusion and Future Work

8.1 Conclusions

This project has shown that performance and architecture analysis can produce complementary improvements in a mature scientific Python codebase. Profiling identified an unused YODA scatter construction as a major cost in the recorded Contur grid workload. Removing it reduced the mean wall-clock time by 35.73 % across three serial runs. The checked `SMBG`, `EXP` and `HLEXP` values matched exactly for all 100 runpoints in the 10×10 parameter grid, giving 300 extracted rows in each of three unprofiled comparisons and one additional profiled comparison.

Static dependency analysis also identified responsibility problems that could be addressed incrementally. The completed architectural changes are the utility import clean-up, the relocation of theory-prediction lookup out of the general utility layer, the move of logarithmic scan spacing to its owning `scan` package, and the separation of static database access into a dedicated package. The relocation of webpage reporting out of `util` has also been merged upstream.

The matched source census found that the five original upward imports from `util` had been removed. Across all imports that conflict with the target structure, however, the count increased from 14 at the baseline revision to 26 at the post-change revision. Moving `static_db` into `database` made sixteen direct accesses from upper packages visible, together with the reverse dependency from `database` to `data`. The increase therefore records unfinished database-boundary work rather than showing that the completed relocations made the design worse.

The merged webpage module still directly accesses `database`. The low-level analysis records are implemented in open merge request `!685`, but domain-object construction and the reverse calls from `data` remain unresolved. The Import Linter contract is retained in draft merge request `!652` as a non-blocking check while these circular dependencies are addressed. The completed changes have clarified where several responsibilities belong and made the remaining dependencies measurable. They have not removed the database–data cycle or enforced the target layering.

These results address the project objectives at different levels of completion. The profiling workflow identified repeated unnecessary work, and the controlled comparison measured its removal while checking the selected statistical outputs. The dependency audit identified unwanted imports and supported several small, merged responsibility relocations. The provisional auto-

mated check and matched source census record the remaining conflicts, but the check is not yet an enforced boundary. The database investigation has characterised the unresolved coupling and tested behaviour that the next change must preserve, without claiming that the final separation has been implemented.

8.2 Future work

The immediate project work is to resolve or mediate the direct database access from `webpages`, complete and review the low-level-record work, and remove the remaining database–data cycle. The same Import Linter contract can then be run on matched intermediate and post-change revisions. This will allow changes in the recorded violations to be considered alongside preserved behaviour and a clearer account of package ownership. Draft merge request `!652` should be merged and its CI job made blocking only after the agreed contract passes.

The remaining database work is to agree and implement the next database-boundary change. The final design should state which layer owns query execution, which component constructs related domain objects, and how existing `Analysis` behaviour is preserved. It should also separate the read-only analysis database from read–write results storage at module level. Concrete class and module names remain provisional.

Longer-term work may consolidate additional SQL currently distributed across data, factory and plotting code behind suitable interfaces. That work is broader than the guaranteed scope of this project and should be presented as a roadmap unless it is actually implemented and tested.

References

[1] A. Buckley et al. "Testing New Physics Models with Global Comparisons to Collider Measurements: The Contur Toolkit". In: *SciPost Physics Core* 4.2 (2021), p. 013. DOI: 10.21468/SciPostPhysCore.4.2.013. arXiv: 2102.04377 [hep-ph].

[2] A. Buckley et al. "Consistent, Multidimensional Differential Histogramming and Summary Statistics with YODA 2". In: *SciPost Physics Codebases* (2025), p. 45. DOI: 10.21468/SciPostPhysCodeb.45. arXiv: 2312.15070 [hep-ph].

[3] C. Bierlich et al. "Robust Independent Validation of Experiment and Theory: Rivet Version 3". In: *SciPost Physics* 8.2 (2020), p. 026. DOI: 10.21468/SciPostPhys.8.2.026. arXiv: 1912.05451 [hep-ph].

[4] Contur Collaboration. *contur package. Contur 3.0.0 documentation*. URL: https://hepcedar.gitlab.io/contur/contur.html#module-contur (visited on 08/06/2026).

[5] J. Rocamonde et al. "Picking the Low-Hanging Fruit: Testing New Physics at Scale with Active Learning". In: *SciPost Physics* 13.1 (2022), p. 002. DOI: 10.21468/SciPostPhys.13.1.002. arXiv: 2202.05882 [hep-ph].

[6] R. C. Martin. *Agile Software Development, Principles, Patterns, and Practices*. Prentice Hall, 2002.

[7] S. Bray. "Profiling and Optimising Contur". MSc dissertation. University College London, 2021.

[8] Python Software Foundation. *The Python Profilers*. URL: https://docs.python.org/3/library/profile.html (visited on 08/04/2026).

[9] *SnakeViz: A Web Browser Based Python Profile Viewer*. URL: https://jiffyclub.github.io/snakeviz/ (visited on 08/04/2026).

[10] J. Fonseca. *gprof2dot: Generate a Dot Graph from the Output of Several Profilers*. URL: https://github.com/jrfonseca/gprof2dot (visited on 08/04/2026).

[11] Python Utilities Authors. *line_profiler: Line-by-Line Profiling for Python*. URL: https://github.com/pyutils/line_profiler (visited on 08/04/2026).

[12] Pylint Project. *cyclic-import / R0401*. URL: https://pylint.readthedocs.io/en/latest/messages/refactor/cyclic-import.html (visited on 08/04/2026).

[13] Pylint Project. *Pyreverse*. URL: https://pylint.readthedocs.io/en/latest/additional_tools/pyreverse/ (visited on 08/04/2026).

[14] B. O. Aakra. *pydeps: Python Module Dependency Graphs*. URL: https://github.com/thebjorn/pydeps (visited on 08/04/2026).

[15] D. Seddon. *Import Linter Documentation*. URL: https://import-linter.readthedocs.io/en/latest/ (visited on 08/04/2026).

[16] E. Hieatt and R. Mee. *Repository*. Mar. 5, 2003. URL: https://martinfowler.com/eaaCatalog/repository.html (visited on 08/04/2026).

Appendix A

Import Linter Violation Snapshot

The original Import Linter CI snapshot summarised in table 4.2 reported the following 14 direct imports. They are reproduced here to keep the main discussion focused on package-level patterns. Source-line numbers refer to commit `c88cd32e61` on the `importlinter-config` branch.

```
contur.scan.grid_tools -> contur.run.run_batch_submit (l.10)
contur.util.rst_utils -> contur.factories.yoda_factories (l.23)
contur.data.data_access_db -> contur.scan.grid_tools (l.6)
contur.data.data_access_db -> contur.scan.os_functions (l.7)
contur.data.build_covariance -> contur.factories.yoda_factories (l.95)
contur.data.sm_theory_builders -> contur.factories.yoda_factories (l.3)
contur.util.file_readers -> contur.data.static_db (l.10)
contur.util.rst_utils -> contur.data (l.3)
contur.util.utils -> contur.data (l.16)
contur.factories.likelihood -> contur.plot.profile_plot (l.27)
contur.factories.test_observable -> contur.plot.yoda_plotting (l.13)
contur.factories.depot -> contur.plot.yoda_plotting (l.16)
contur.data.data_objects -> contur.plot.html_utils (l.8)
contur.util.rst_utils -> contur.run.arg_utils (l.231)
```

This is a diagnostic snapshot of a broken, non-blocking contract. It is not the final post-refactoring result.

Appendix B

Cross-Package Import Census

The complete census script is stored as `experiments/architecture/count_package_imports.py`. It uses only the Python standard library and reads source files directly from a specified Git revision. The generated CSV and JSON files are stored under `experiments/architecture/package-import-counts/`.

The two recorded commands selected the following revisions:

```
python count_package_imports.py \
  --repo <contur-repository> \
  --revision cb0d0f1866 \
  --label baseline-cb0d0f1866 \
  --output-dir package-import-counts

python count_package_imports.py \
  --repo <contur-repository> \
  --revision cf6ca9b157 \
  --label post-change-cf6ca9b157 \
  --output-dir package-import-counts
```

For each revision, the script records every counted source location, the complete package matrix, the directions that conflict with the target and a JSON summary of the counting rules. The baseline run parsed 64 Python files and counted 167 cross-package direct imports. The post-change run parsed 66 files and counted 170. Neither run had a parse failure. The baseline classification reproduced the 14 imports across eight directions in the recorded Import Linter CI snapshot.

Glossary of Terms

Continuous integration (CI). Automated jobs that build and test a proposed code change.

Contur. A Python package that tests predictions from beyond-the-Standard-Model physics against published collider measurements.

Dependency contract. A machine-readable rule describing which packages may import other packages. Import Linter checks the source tree against these rules.

Domain object. A software object representing a concept in the problem domain, such as an analysis, experiment or pool.

Import Linter. A static-analysis tool used here to check package imports against the proposed dependency contracts.

Merge request (MR). A proposed set of source-code changes submitted for review before it is merged into the main branch.

Profiling. Measurement of where execution time is spent within a program.

Rivet. The analysis framework that provides routines and reference data representing published collider measurements. Contur processes the outputs of these routines.

Runpoint. One model-parameter point in a Contur grid and the associated input data.

Statistical output modes. `SMBG` uses an available Standard Model prediction as the background. `EXP` is the expected limit obtained by moving the data central value to that prediction. `HLEXP` applies the expected-limit calculation with uncertainties scaled for projected High-Luminosity Large Hadron Collider data.

Wall-clock time. The elapsed real time from the start of a process to its completion, including time spent in called functions and system operations.

YODA. The histogramming library used by Contur and Rivet to store histogram and scatter data.